

```

Node reverseK( head, k) {
    int k1 = k;

    curr = head;
    prev = null;
    cn = null;

    while( curr != null && k1 > 0) {
        cn = curr->next;
        curr->next = p;
        p = curr;
        curr = cn;
        k1--;
    }

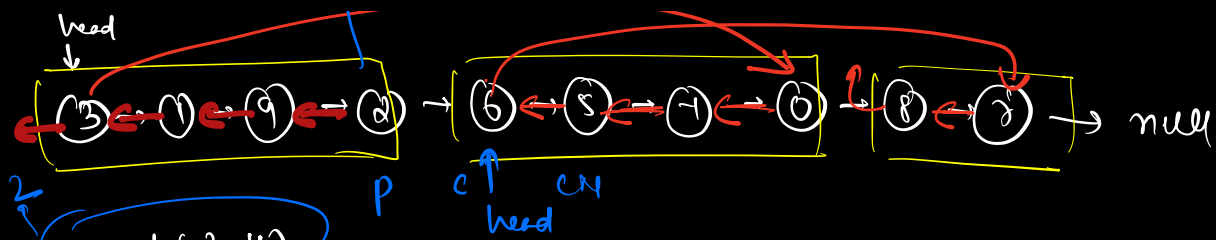
    head->next = reverseK( curr, k);

    return prev;
}

```

$2 \rightarrow 9 \rightarrow 1 \rightarrow 3 \rightarrow 0 \rightarrow 7 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow \text{null}$

head



CN

reverse(3, 4)

head 3

3.next = reverse(6, 4)

p = null

c = 6

CN = c.next

6.next = reverse(8, 4)

p = null

c = 8

CN = c.next

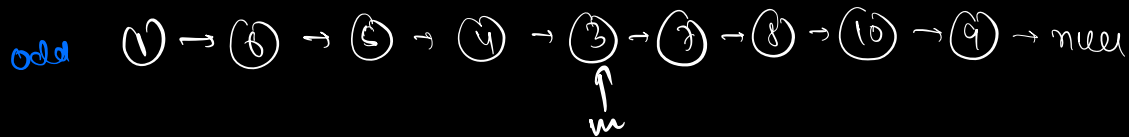
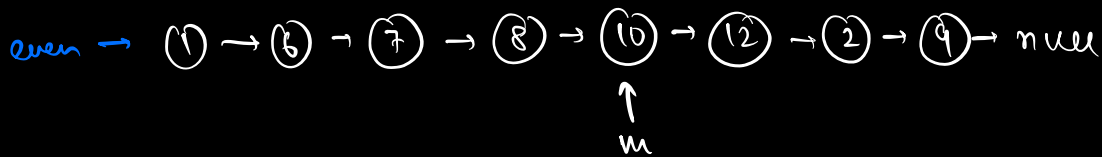
$T \Rightarrow O(N)$

$SC \Rightarrow O(N/k)$

$$N \text{ odd} \rightarrow \frac{n+1}{2}$$

$$N \text{ even} \rightarrow \frac{n}{2} + 1$$

1: Find middle of a LL:-

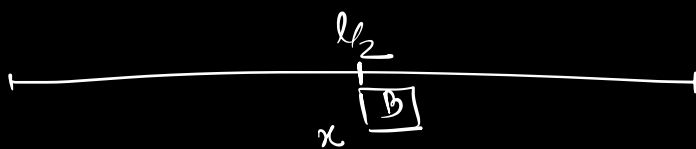
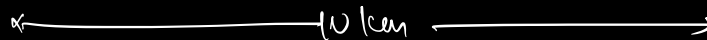
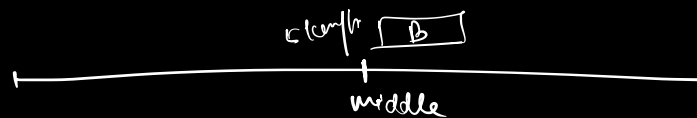
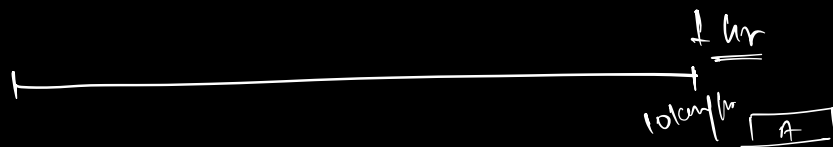


Basic

- (i) Find length of LL $\rightarrow O(N)$
- (ii) Iterate to middle element $\rightarrow O(N/2)$

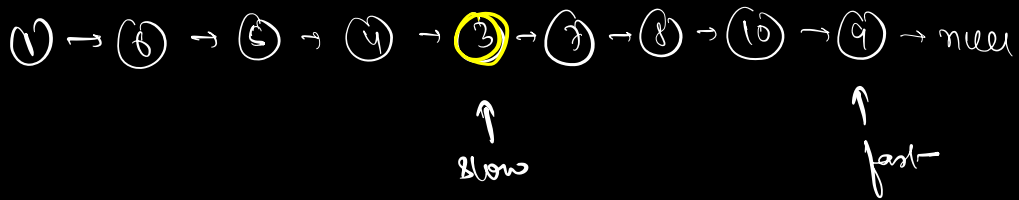
(1)

TC $\Rightarrow O(N) \rightarrow 2 \text{ iterations}$



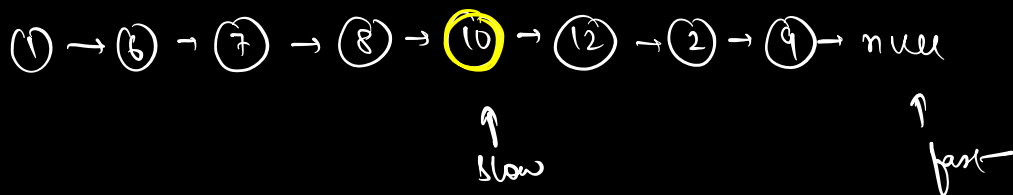
time taken by A to reach end $\Rightarrow \frac{l}{2u}$

how much dist will B cover in $\frac{l}{2u} \rightarrow \frac{l}{2u} \times u = \frac{l}{2}$



$slow = slow.next$

$fast = fast.next.next$



```
if (head == null || head.next == null)
    return head;
```

$slow = head, fast = head$

while ($fast \neq null$ & $fast.next \neq null$) {

$slow = slow.next$

$fast = fast.next.next$;

}

return slow;

TC $\Rightarrow O(N) \leftarrow 1 \text{ iteration}$

Q2. Merge 2 LL

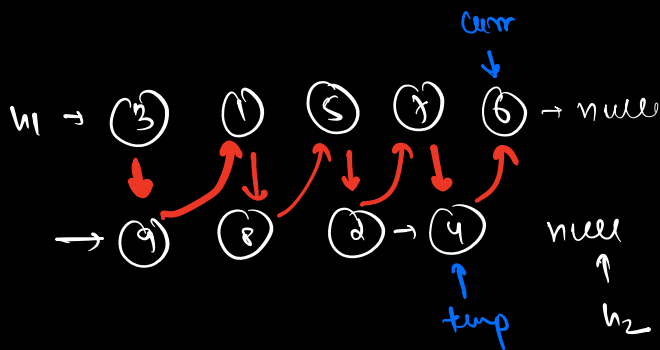
$\text{length}(h1) > \text{length}(h2)$

$h1 \rightarrow (3) \rightarrow (1) \rightarrow (5) \rightarrow (7) \rightarrow (6) \rightarrow \text{null}$

$h2 \rightarrow (9) \rightarrow (8) \rightarrow (2) \rightarrow (4) \rightarrow \text{null}$

H.W
[if we don't know
the length, solve]

OP $\Rightarrow (3) \rightarrow (9) \rightarrow (1) \rightarrow (8) \rightarrow (5) \rightarrow (2) \rightarrow (7) \rightarrow (4) \rightarrow (6) \rightarrow \text{null}$



while ($h2 \neq \text{null}$) {
 $\text{temp} = h2$

$h2 = h2.\text{next}$

$\text{temp}.\text{next} = \text{curr}.\text{next}$

$\text{curr}.\text{next} = \text{temp}$

$\text{curr} = \text{temp}.\text{next}$

}

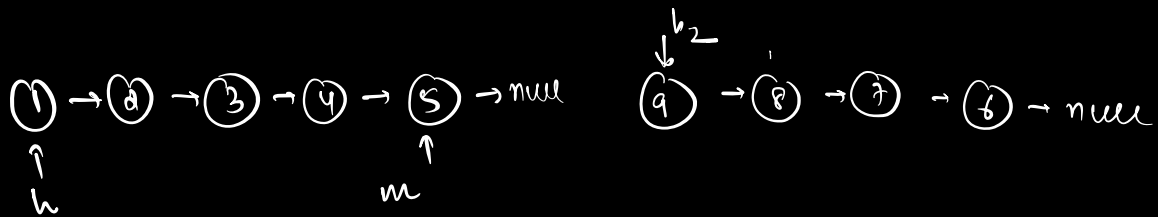
return $h1$.

TC $\Rightarrow O(N+N)$ SC $\Rightarrow \underline{O(1)}$

Q 3. Reorder a LL

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow \text{null}$

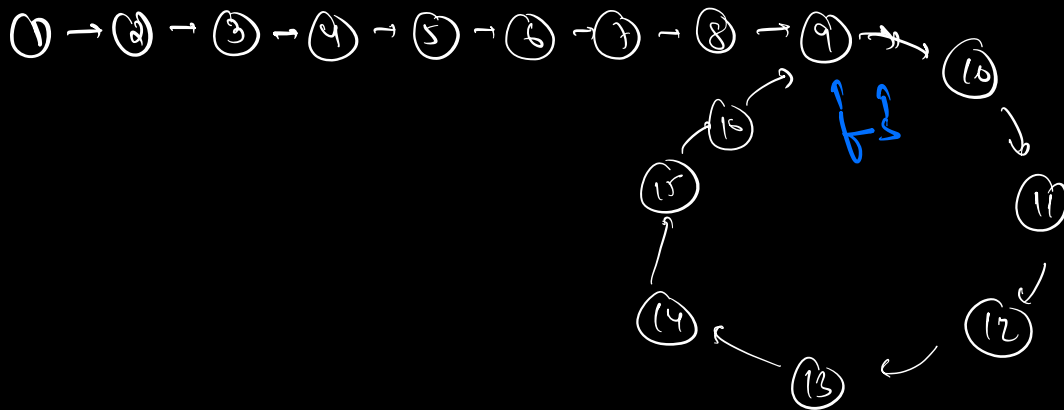
op $\Rightarrow 1 \rightarrow 9 \rightarrow 2 \rightarrow 8 \rightarrow 3 \rightarrow 7 \rightarrow 4 \rightarrow 6 \rightarrow 5 \rightarrow \text{null}$



$\left\{ \begin{array}{l} \text{Node } m = \text{middle}(\text{head}); \\ h_2 = m.\text{next}; \\ m.\text{next} = \text{null}; \\ h_2 = \text{reverse}(h_2); \\ \text{return } \text{merge}(h, h_2) \end{array} \right.$

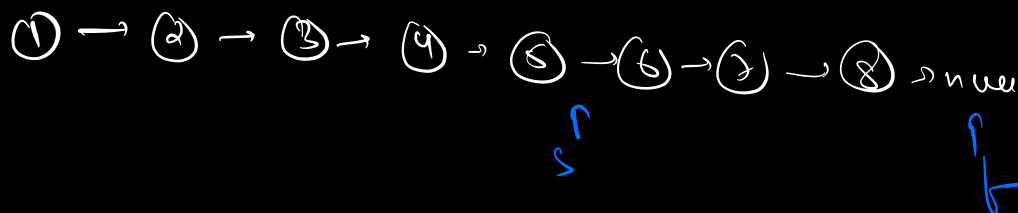
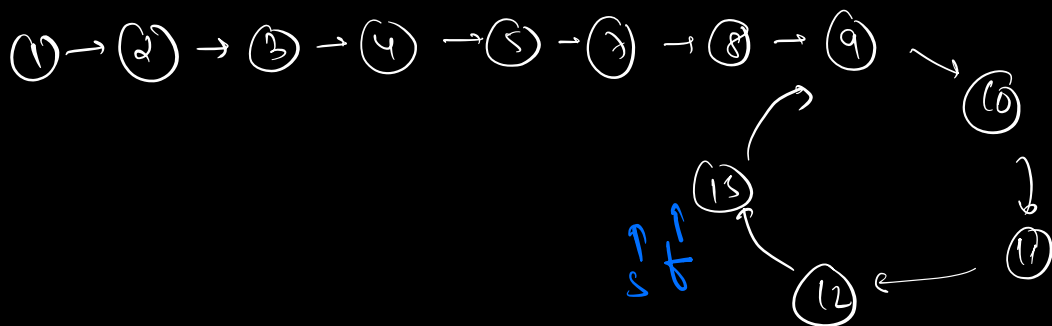
$\boxed{\begin{array}{l} TC \approx O(N) \\ SC \approx O(1) \end{array}}$

Q Detect if there is a cycle in LL



1) Hashmap / Hashset

TC $O(N)$ | SC $O(N)$



slow, fast $\rightarrow$ if these two ptr.
ever meet, then there
is a loop definitely

slow = head

fast = head

while (fast != null & fast.next != null) {

slow = slow.next

fast = fast.next.next

if (slow == fast)

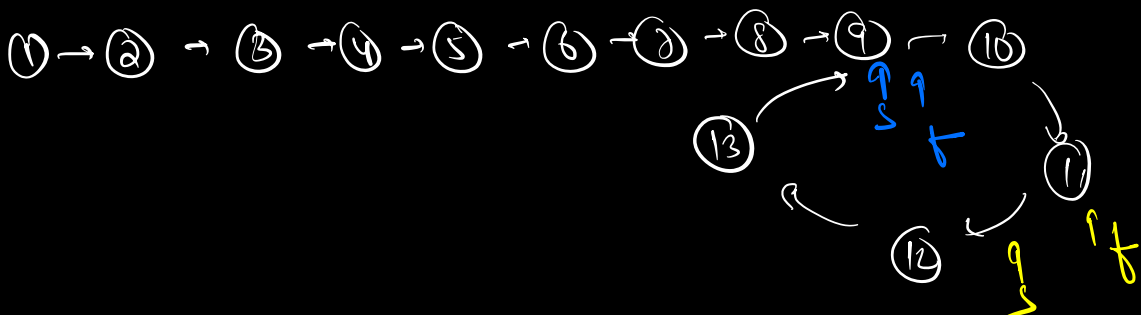
return true.

}

return false;

}

$\rightarrow$ Starting point of cycle:



1) take slow & fast ptr.

if they meet $\rightarrow$ cycle [detect cycle]

$\downarrow$

* take slow ptr. to start of LL

* move both ptr at $\uparrow$ speed,

* wherever they meet is

start of cycle

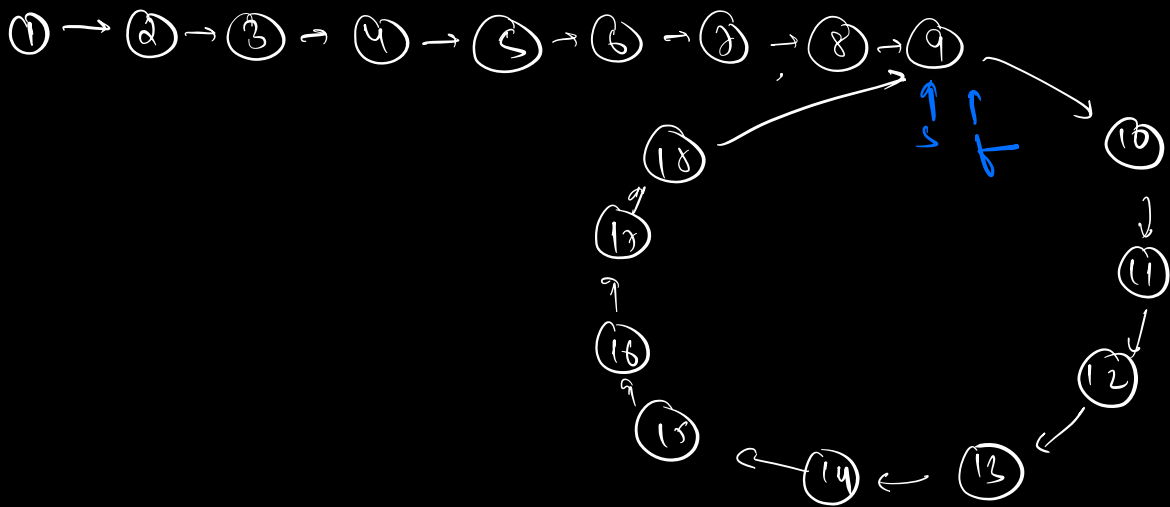
slow = head

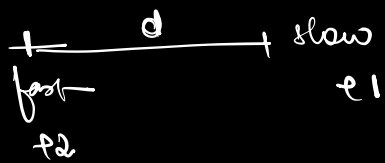
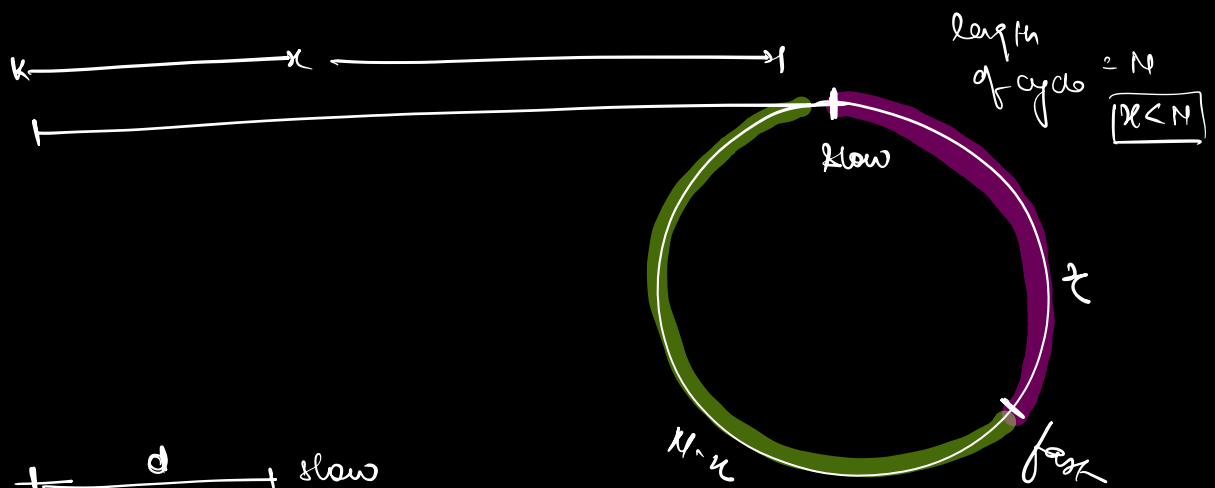
while (slow $\neq$ fast) {

slow = slow.next

fast = fast.next

} return slow $\leftarrow$ start of cycle

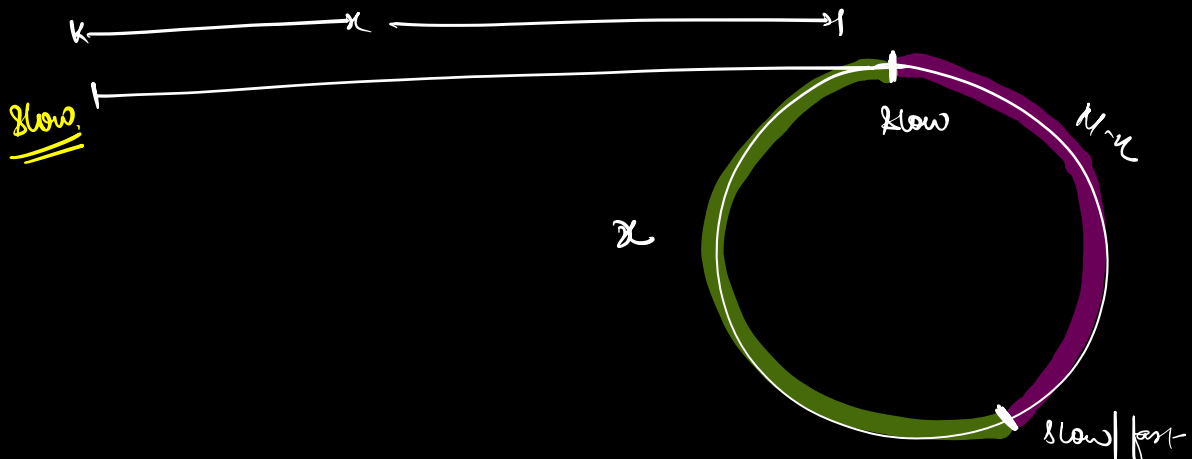




So, at each step
 fast and slow are
 moving closer by 1 unit

So for d dist, they d iterations / steps

slow and fast
 will meet after $N-k$
 iterations

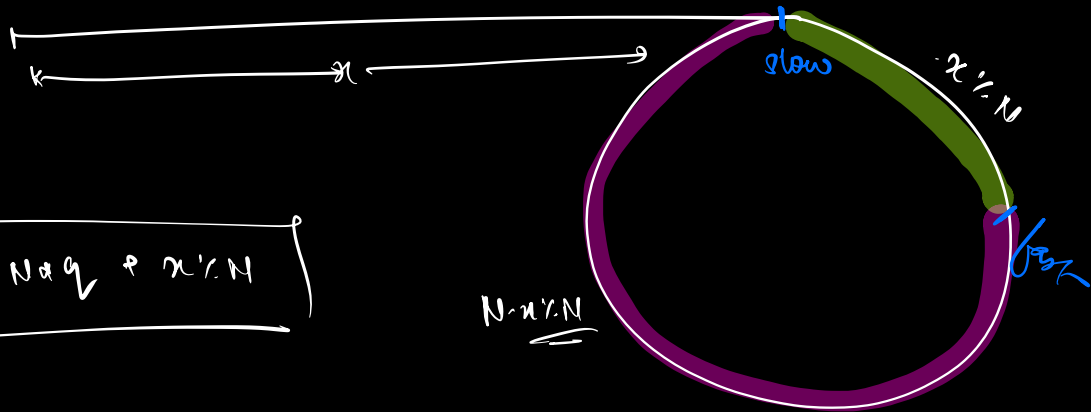


length of cycle = N

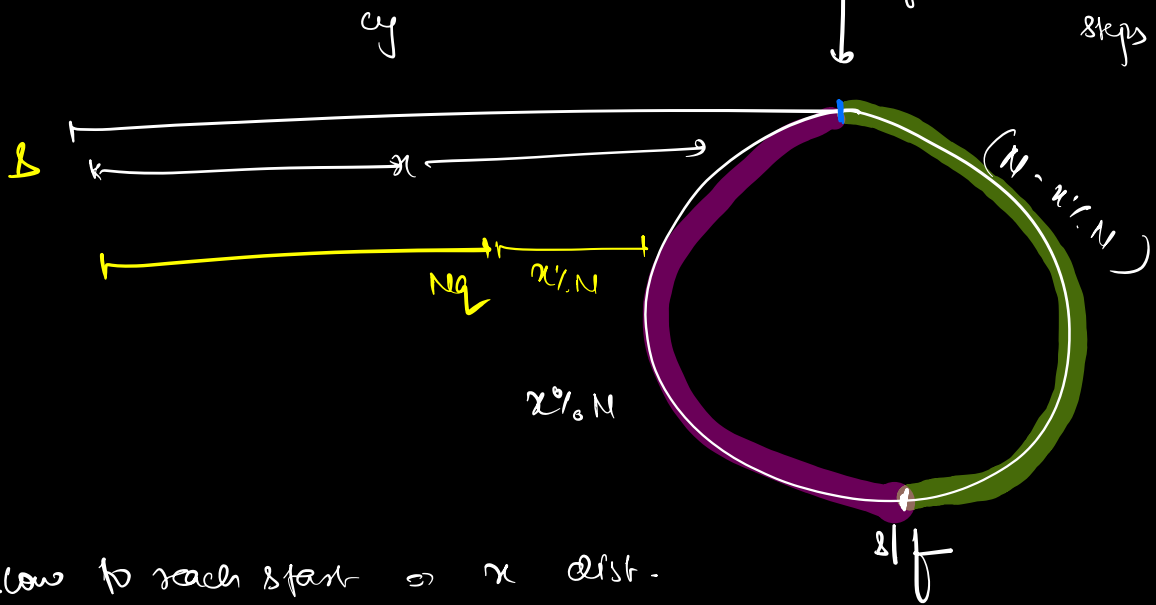
$$\underline{x \geq N}$$

$$x = Naq + x \% N$$

$$N - x \% N$$



they will meet
after $N - x \% N$
steps



slow to reach start $\Rightarrow x$ dist.

so fast will also move $\Rightarrow x$ dist.

$$x = \underline{Naq} + x \% N$$

$\downarrow$
same position

also moved Naq dist.

removing

$x' \cdot y \longleftrightarrow x' \cdot N$

they meet at start

1) take slow and fast ptr.

2) move $\rightarrow$ slow = slow.next

fast = fast.next.next

3) if they meet, proves $\rightarrow$ cycle.

4) move slow to head

5) move slow = slow.next

fast = fast.next

wherever they meet is starting pt.

- How
- i) detect a loop
 - ii) find starting pt. of loop
 - iii) remove the loop.